

pybounds Diagnostic Report — Mono-Camera System

States

g — ground speed

d — distance above ground

Dynamics $\dot{\mathbf{x}} = f(\mathbf{x}, u)$

$$\dot{g} = u$$

$$\dot{d} = 0$$

Measurement $\mathbf{y} = h(\mathbf{x})$

$$r = g/d \quad (\text{ventral optic flow})$$

Simulation parameters

$$\mathbf{x}_0 = [g_0, d_0] = [2.0, 3.0]$$

$$dt = 0.01 \text{ s}$$

$$u(t) = \sin(3t) \text{ (with near-zero segment } 3 \leq t < 6 \text{ s)}$$

Observability parameters

Window size $w = 6$ time-steps

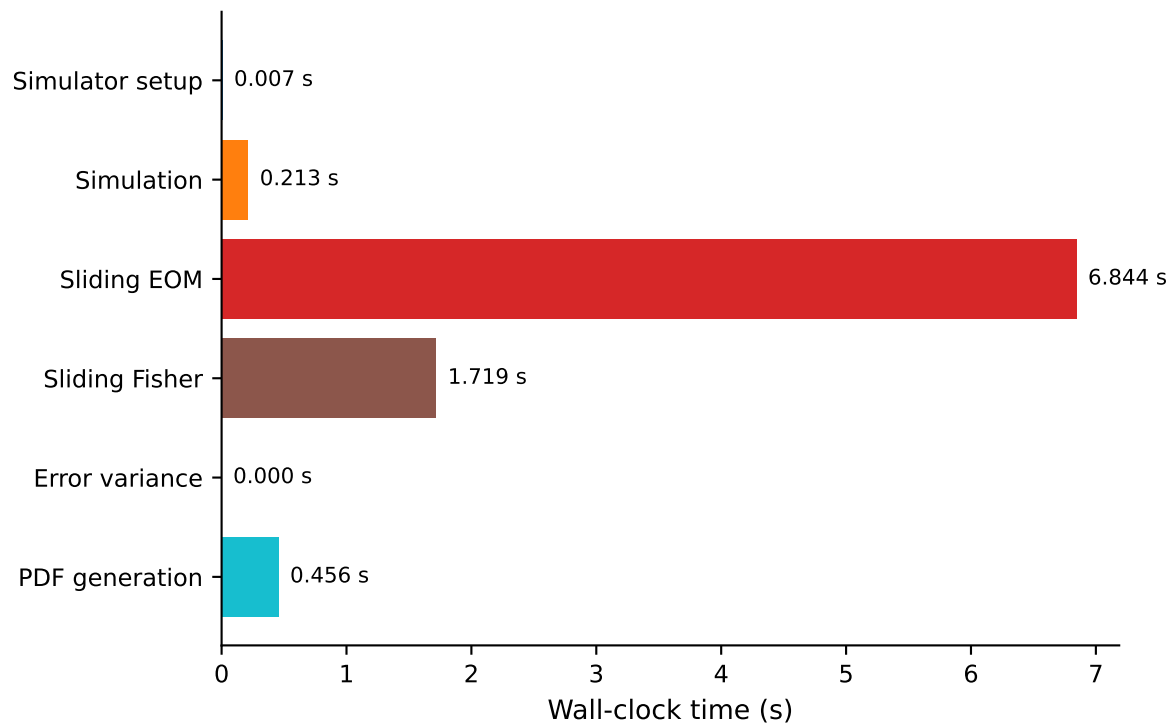
Perturbation $\varepsilon = 10^{-4}$

Sensor noise $R = \{r : 0.1\}$

Regularisation $\lambda = 10^{-8}$

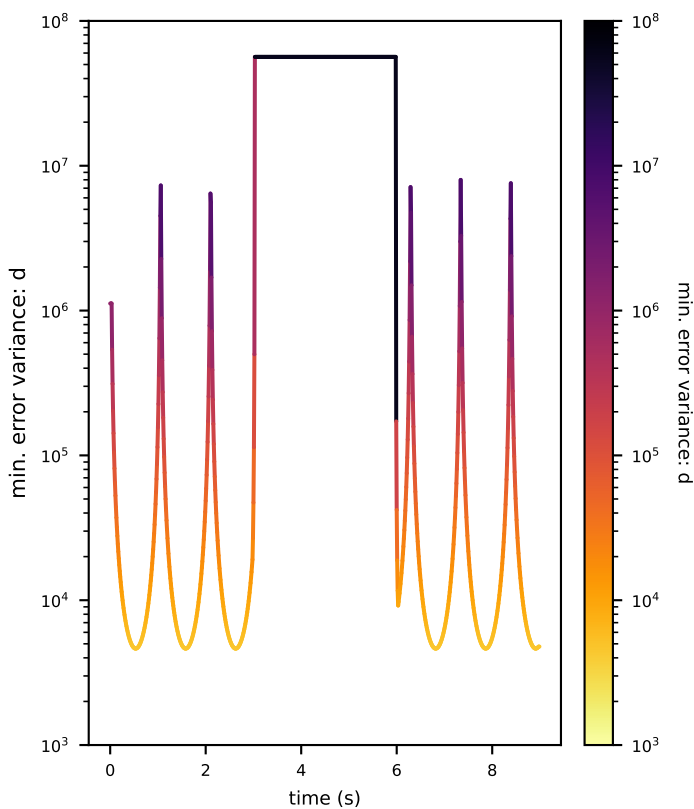
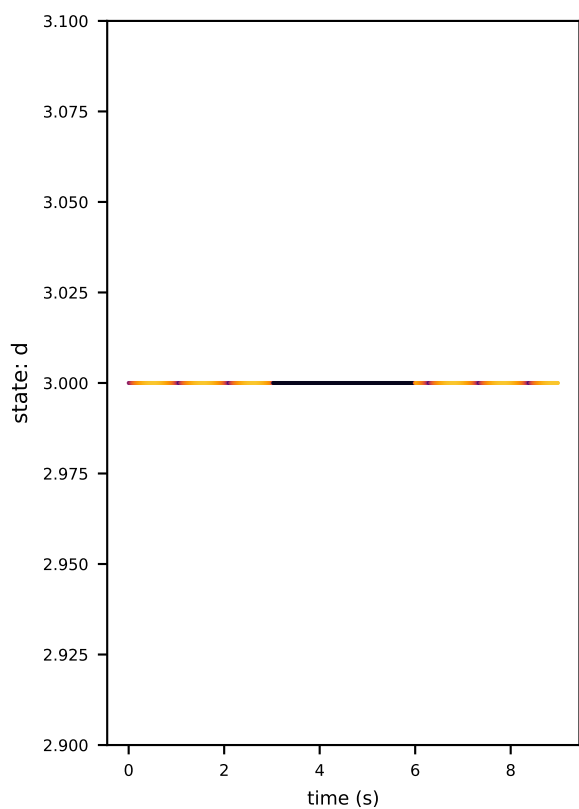
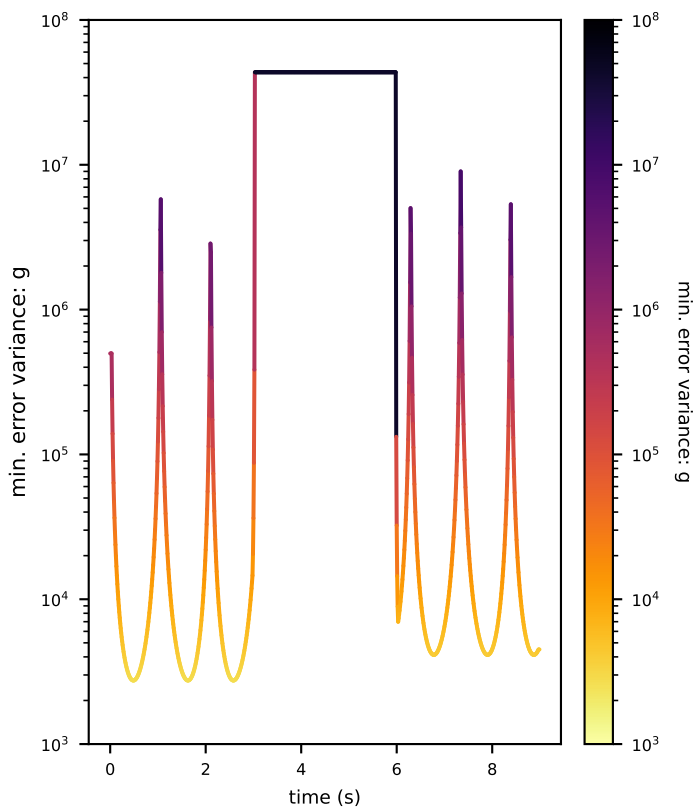
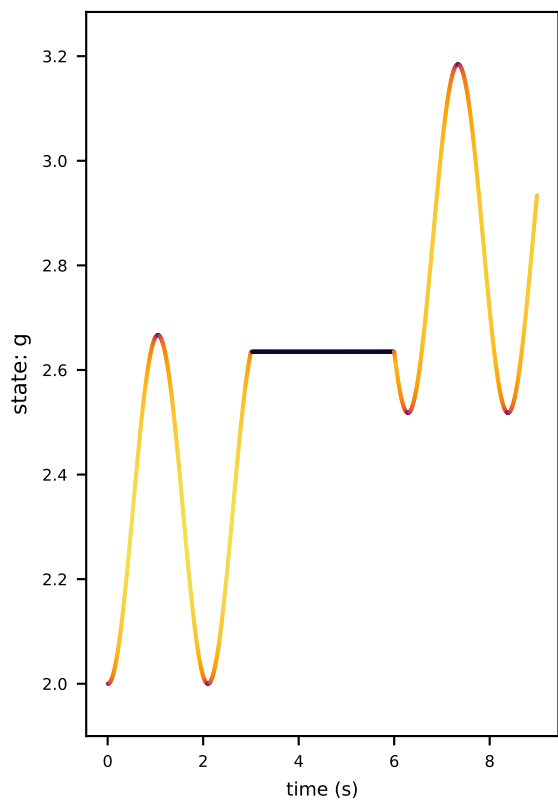
Pipeline Timing Summary

Trajectory: 900 time-steps | Sliding windows: 895 | Window size: 6



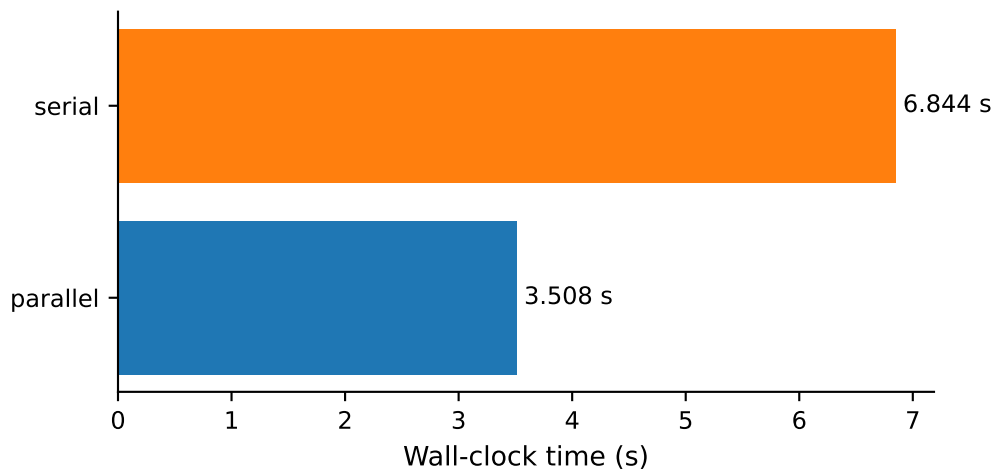
Step	Time (s)	% of total	Description
Simulator setup	0.007	0.1%	Construct Simulator: build do_mpc model, MPC controller
Simulation	0.213	2.3%	Open-loop simulate 900 steps (9.0 s of simulated time)
Sliding EOM	6.844	74.1%	SlidingEmpiricalObservabilityMatrix: 895 windows × 4 simulations each (2 perturbations × 2 states)
Sliding Fisher	1.719	18.6%	SlidingFisherObservability: $F = O^T R^{-1} O$ and $(F + \lambda I)^{-1}$ for 895 windows
Error variance	0.000	0.0%	get_minimum_error_variance() + NaN fill
PDF generation	0.456	4.9%	Render equations page, timing page, observability plot
TOTAL	9.240	100%	

Minimum Error Variance over Time (mono-camera system)



Parallel vs Serial Benchmark

SlidingEmpiricalObservabilityMatrix | 895 windows | parallel_sliding=True | simulator_factory=<callable> | n_workers=18 (multiprocessing.Pool, spawn)



Serial time: 6.844 s

Parallel time: 3.508 s

Speedup: 1.95×

Correctness: PASS — results match serial

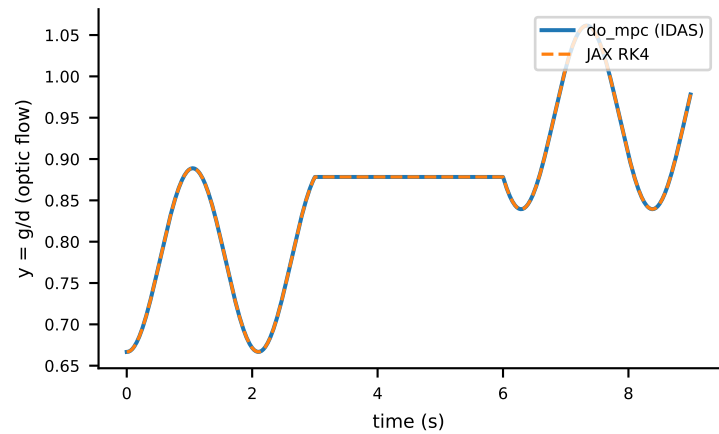
Notes

- Simulator.simulate() mutates shared state (self.simulator, self.x, ...) on each call.
CasADi/IDAS is NOT thread-safe: ThreadPoolExecutor causes fatal crashes.
- The new simulator_factory parameter passes a zero-arg callable to SEOM.
Each worker process calls factory() once to get its own Simulator.
This eliminates all shared state between workers.
- Uses multiprocessing.Pool with start method "spawn" (macOS/Windows default).
"spawn" avoids fork-safety issues with CasADi shared libraries.
- Each worker incurs ~0.02 s Simulator setup overhead (one-time, amortized across all windows assigned to that worker).

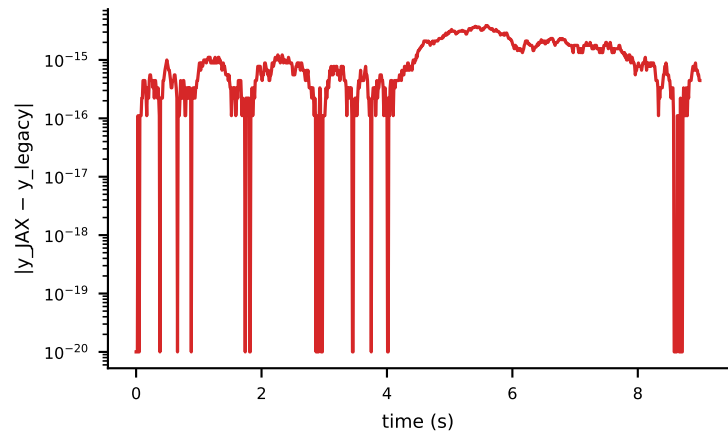
JAX Autodiff vs Legacy Numerical Observability

Mono-camera system | RK4 integrator | jacfwd Jacobian vs $\pm\epsilon$ perturbations

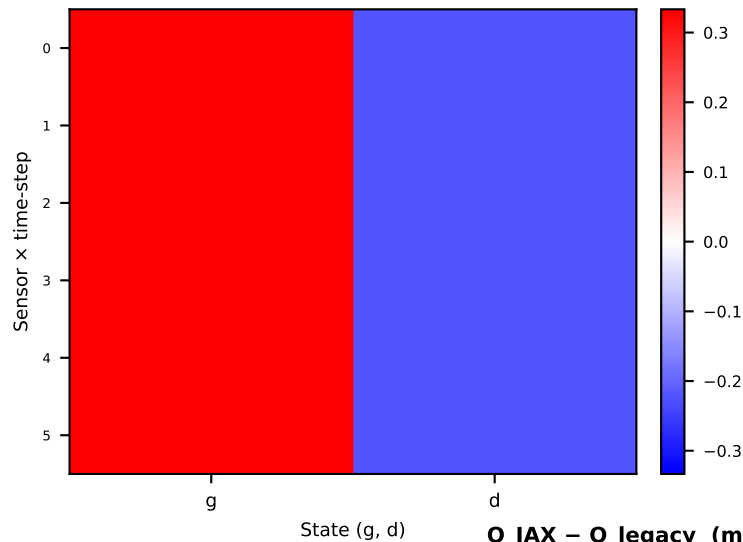
Simulation output: y trajectory



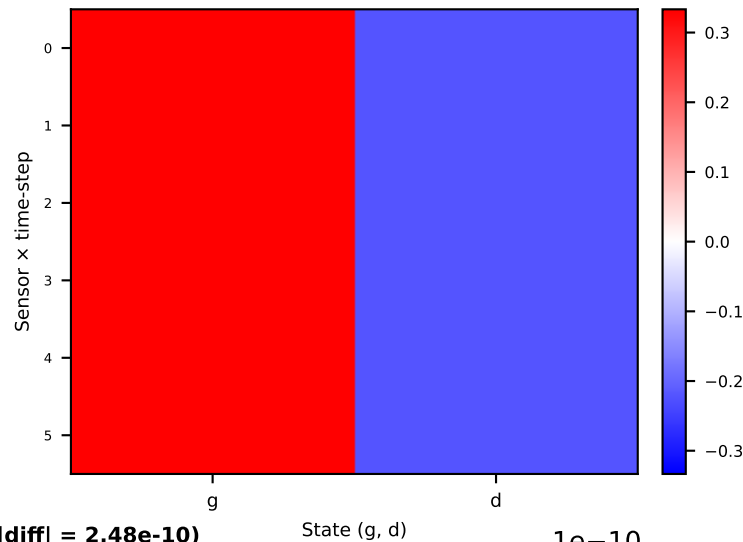
Residual (log scale)



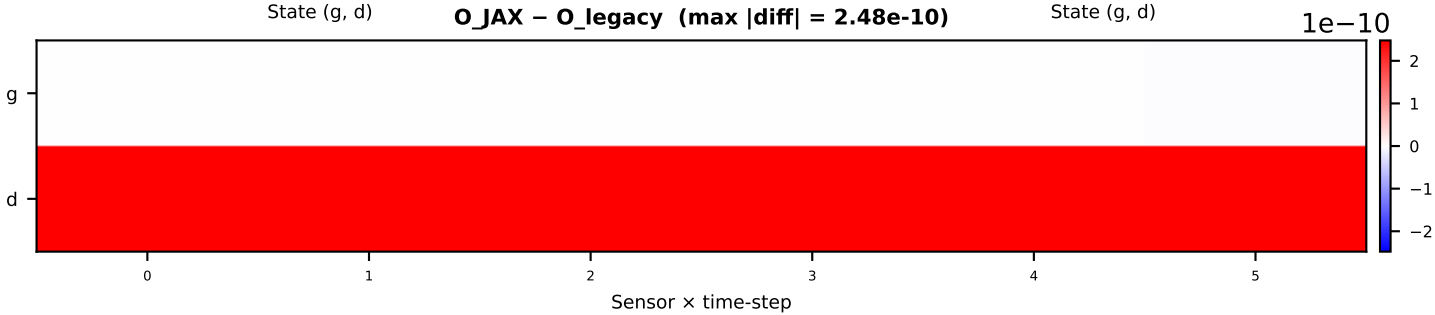
O legacy (numerical $\pm\epsilon$)



O JAX (autodiff jacfwd)



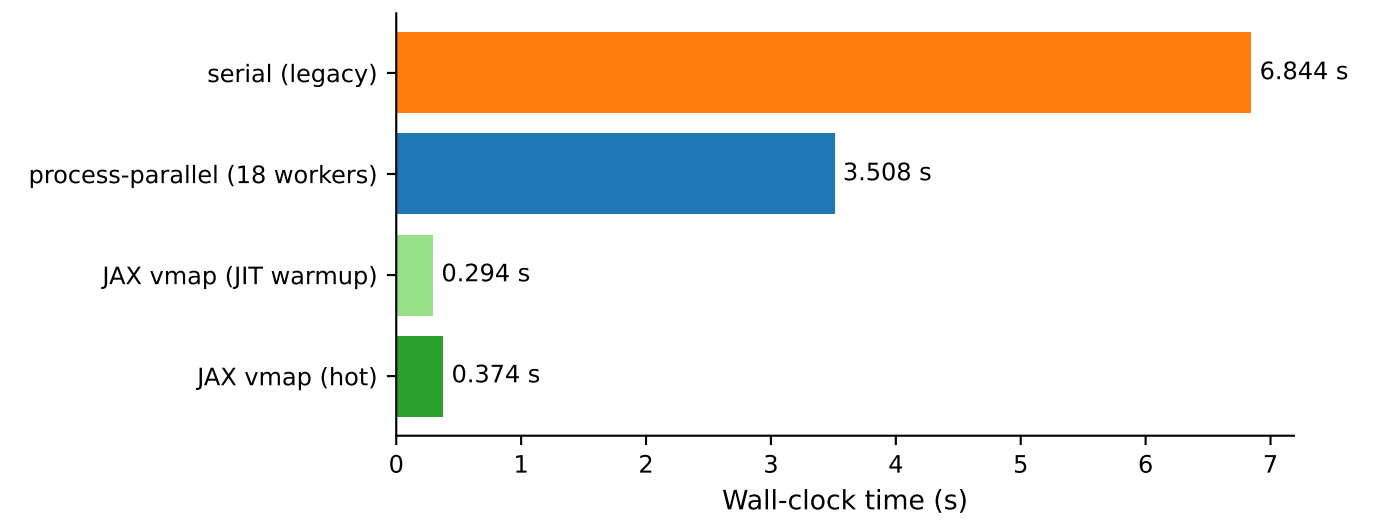
O_JAX - O_legacy (max |diff| = 2.48e-10)



Step	Time (s)	Speedup vs legacy	Notes
do_mpc simulate (full traj)	0.213	1.0× (baseline)	900 steps, IDAS solver
JAX RK4 simulate (JIT warmup)	0.072	3.0×	First call: traces + compiles XLA kernel
JAX RK4 simulate (hot)	0.0222	9.6×	Subsequent calls: runs compiled kernel
Legacy O, single window	0.0076	1.0× (baseline)	4 perturbation sims (2 states × $\pm\epsilon$)
JAX O = jacfwd (JIT warmup)	0.066	0.1×	First call: traces jacfwd + compiles
JAX O = jacfwd (hot)	0.0578	0.1×	Hot call; max O_diff = 2.48e-10

JAX vmap Sliding Observability Benchmark

SlidingEmpiricalObservabilityMatrix | 895 windows | window size: 6



Serial (legacy):	6.844 s	1.0× (baseline)
Process-parallel (18w):	3.508 s	2.0×
JAX vmap (JIT warmup):	0.294 s	23.3×
JAX vmap (hot):	0.3745 s	18.3×
Correctness:	PASS — max O_jax − O_serial = 3.95e-10	

How it works

- JaxSlidingEmpiricalObservabilityMatrix extracts all window initial states and input sequences into two batched arrays:

x0_batch

shape (n_windows, n)

— one x0 per window

u_batch

shape (n_windows, w, m)

— one input sequence per window

- A single vmapped+JIT-compiled jacfwd call computes all Jacobians at once:

vmapped_jac

= jax.jit(jax.vmap(jax.jacfwd(sim, argnums=0)))

jac_batch

= vmapped_jac(x0_batch, u_batch)

(n_windows, w, p, n)

- On CPU this becomes a single parallelisable XLA computation; on GPU it would map to a single kernel launch, giving further speedups.
- No process spawning, no Simulator cloning — pure JAX array operations.
- JIT warmup cost is paid once; all subsequent calls use the compiled kernel.